VOLMIND

Trade the divergence between AI belief and market belief.

An autonomous, multi-agent options research and trading terminal built on Alpaca — an independent probability estimate, checked against the market's own pricing, cleared by diligence and risk review, then executed and managed end to end.

LANGGRAPH AGENT PIPELINE · ALPACA OPTIONS + MARKET DATA · PAPER TRADING · FAIL-CLOSED

THE IDEA

Two independent opinions. One trade only when they disagree enough to matter.

AI BELIEF

An independent probability estimate

News, Fundamental, and Probability agents each call an LLM over live Alpaca market and news data and return a structured, evidence-required forecast — a claim with no cited evidence has its confidence automatically clamped.

−

divergence

MARKET BELIEF

What options pricing already implies

A market-implied probability is computed directly from the live Alpaca option chain — delta-approximated, near-the-money — completely independent of the LLM.

THRESHOLD

A thesis is only interesting once **both** the divergence and the AI's confidence clear configured thresholds — then it still has to survive an independent diligence review before it's ever priced as a trade.

ARCHITECTURE

A 10-node LangGraph pipeline — not a black box

BELIEF FORMATION

Scout → News → Fundamental → Probability → Mkt-Implied Prob. → Divergence

↓

GATES & EXECUTION

Diligence [gate] → Options Architect → Risk [gate] → Execution → Evaluation

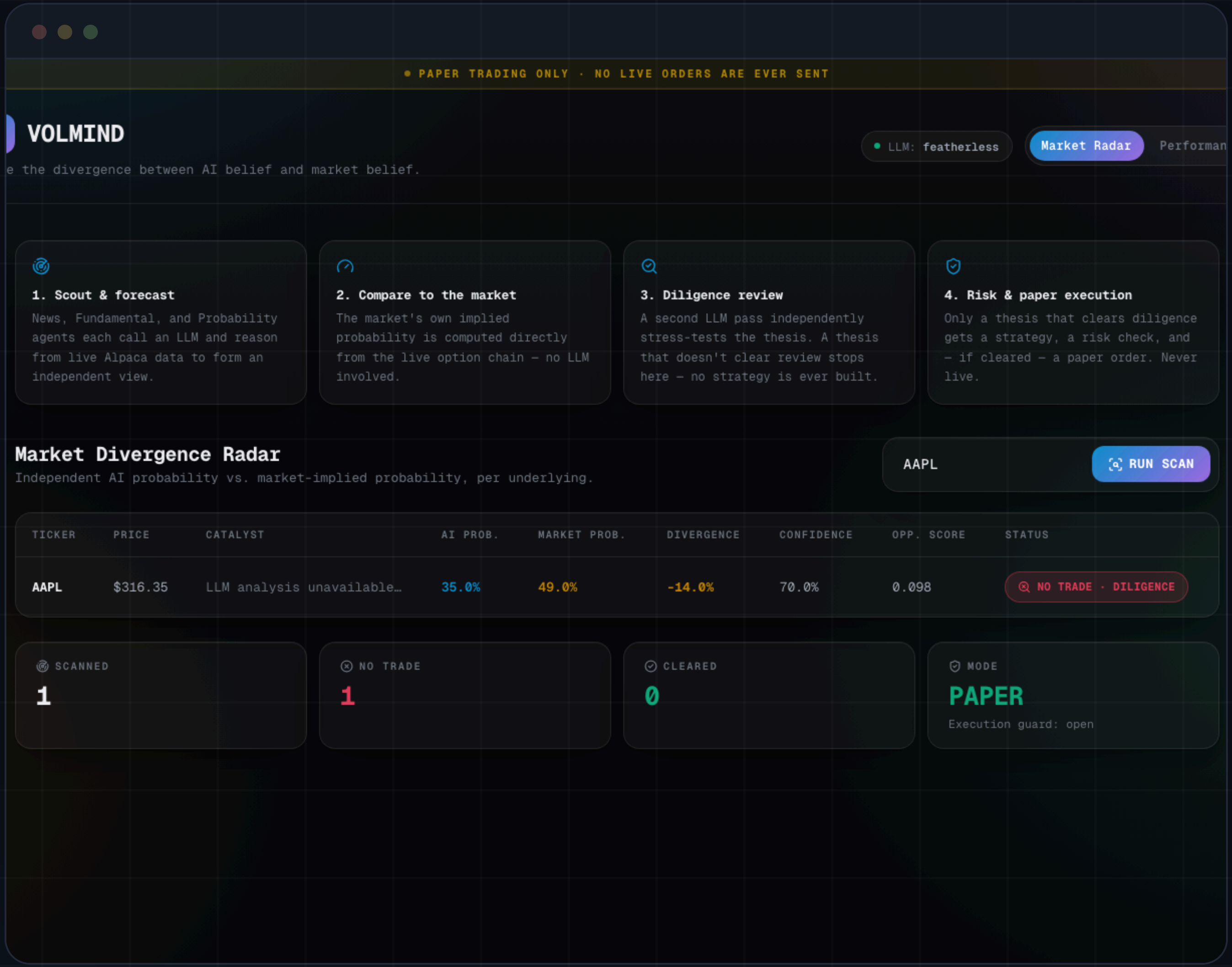
↓

POSITION LIFECYCLE

Position Monitor (take-profit / stop-loss / expiry) → Close → Realized P&L

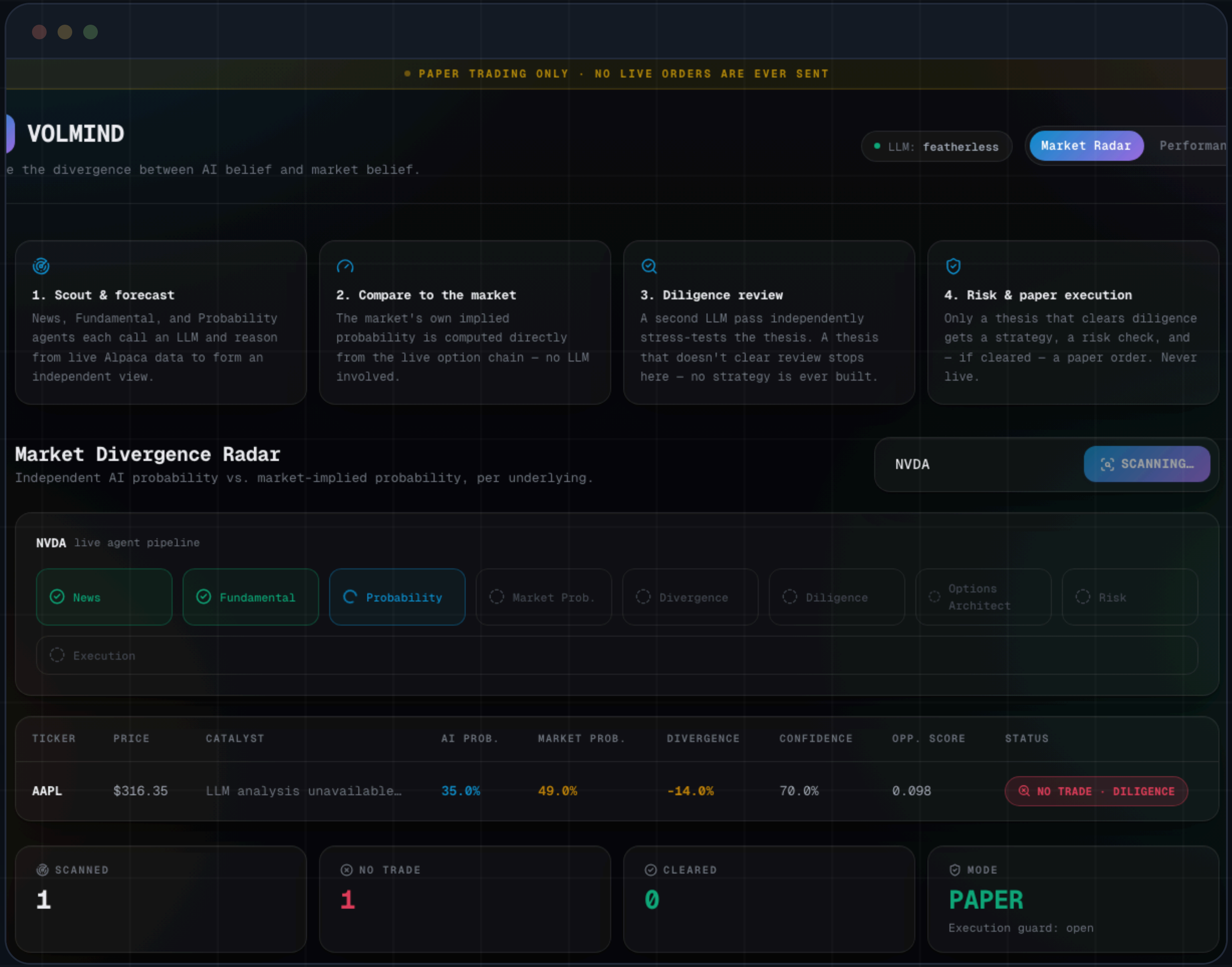
A thesis that fails **Diligence** or **Risk** halts the graph right there — no strategy is priced, no order is submitted. Every node's input, output, confidence, and reasoning is logged with a trace ID, so a blocked trade always has an inspectable paper trail.

Market Radar — every ticker's divergence, at a glance



- → Watchlist ranked by **AI conviction vs. market-implied probability** — the widest, most confident gaps surface first.
- → Every row is a live pull from Alpaca market and options data, not a static mock.
- → One click opens the full agent trail behind any ticker's number.

Real progress, not a spinner

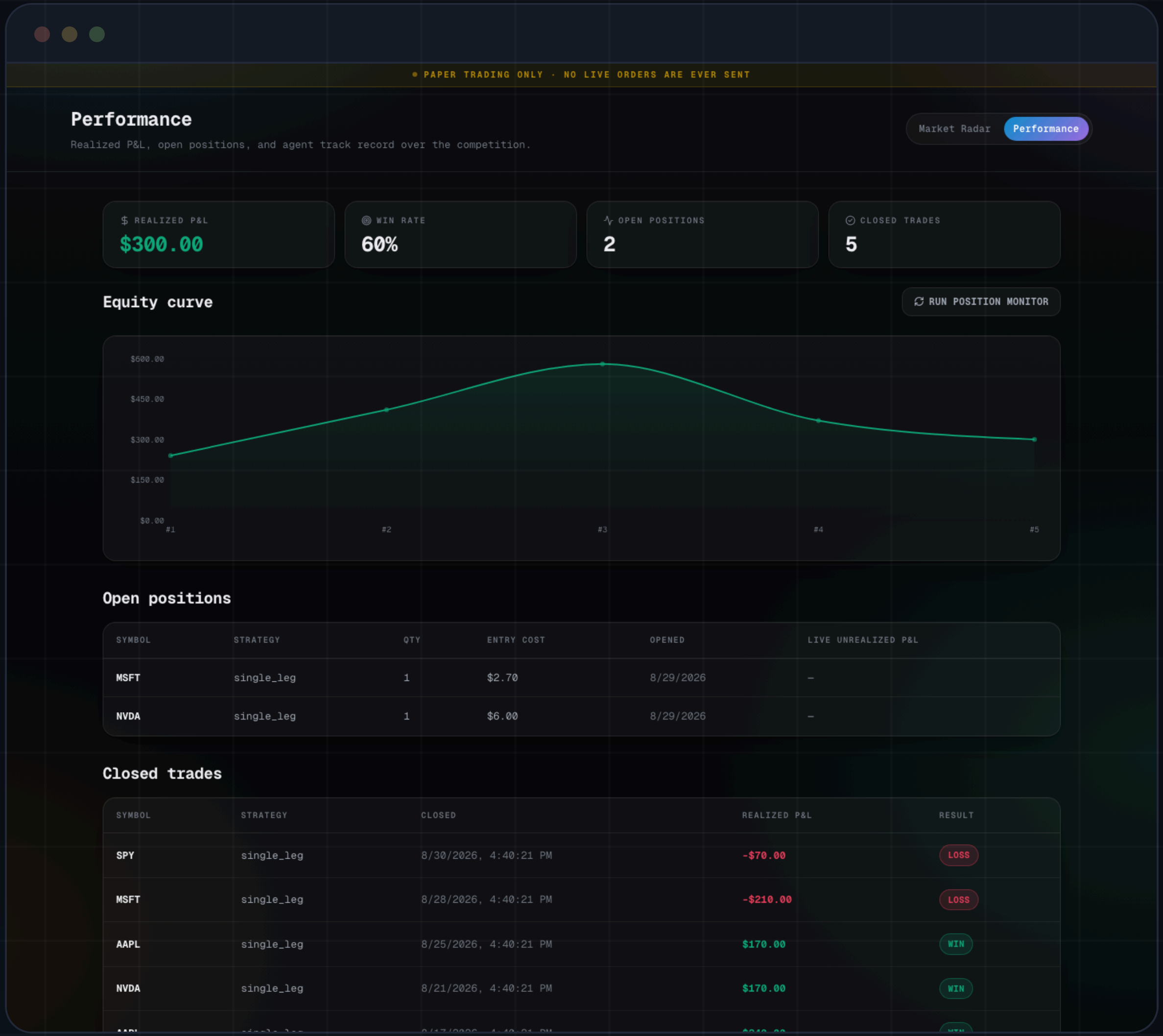


- The backend streams a server-sent event the instant each agent **actually** finishes (`GET /scan/stream`).
- The stepper reflects genuine pipeline state — including steps that never ran because diligence review flagged the thesis first.
- Greyed-out steps mean "skipped by a gate," never "stuck pending."

Agent Council & Diligence Review — a fully inspectable trail

- → Every belief-forming agent's reasoning, cited evidence, and confidence — side by side, not hidden behind a single score.
- → The **Diligence Agent's** independent review: concerns raised, a concern score, and the verdict that decides whether a trade is even proposed.
- → Nothing here is post-hoc — it's the same trail the pipeline used to decide.

Performance — realized P&L, open positions, agent reputation



- → The Position Monitor is the only thing that writes `realized_pnl` — this curve is earned, not simulated.
- → Every closed trade updates a **Brier-scored track record** for the five belief/gating agents against real outcomes.
- → "Run Position Monitor" triggers a mark-to-market and exit-rule check on demand.

SAFETY

Fail-closed by design — five independent layers

01 · DILIGENCE GATE

An independent second LLM pass plus deterministic floor checks the model can't argue past. Failing either halts the graph — no proposal, no risk check, no order.

02 · RISK AGENT

Three independent vetoes checked fresh every time: max concurrent open positions, a daily realized-loss circuit breaker, and per-trade max loss.

03 · PAPER-TRADE GUARD

An exact-string `ALPACA_PAPER_TRADE` check before every order — opening **or** closing — plus `TradingClient` hardcoded to paper at construction as defense in depth.

04 · POSITION MONITOR

Every open position marked to market each cycle; the first rule to trip — expiry cutoff, take-profit, stop-loss — closes it automatically.

05 · MARKET-HOURS GATE

The autonomous scheduler is opt-in and off by default. Every cycle checks Alpaca's own clock first — it never fires against a closed tape.

10	5	3	33
PIPELINE NODES	AGENTS SCORED ON P&L	INDEPENDENT RISK VETOES	TESTS PASSING

INFRASTRUCTURE

Built on Alpaca's live infrastructure, end to end

TRADING

Orders & positions

Risk-cleared theses submit as real paper orders via `TradingClient`. Multi-leg (`OrderClass.MLEG`) already wired for future strategies.

OPTIONS DATA

Full live chain

`OptionChainRequest` pulls bid/ask/IV/Greeks per OCC contract; positions are re-quoted every monitor cycle.

MARKET & NEWS

No synthetic data

Live underlying quotes drive discovery; real Alpaca news with citations feeds the News Agent directly.

CLOCK

Market-hours gate

Alpaca's own `get_clock()` gates every autonomous scan and monitor cycle — never trades a closed tape.

AUTONOMOUS RUNTIME

In-process `APScheduler` jobs inside the FastAPI app, or the same cycle functions as a standalone long-lived process (`scripts/run_autonomous.py`) — built to scan and manage its book unattended for the length of a competition. Trades, predictions, and agent reputation persist to an auditable local ledger.

BEYOND THE HACKATHON

One pipeline. Two deployment modes.

BUILT FOR THIS HACKATHON

Autonomous trading agent

A scheduler triggers scan and position-monitor cycles on their own. Left running, VOLMIND scans its watchlist and manages its full book — open, watch, close — for the length of a competition, with no human in the loop.

THE SAME GRAPH, RETARGETED

Decision-support terminal

Point the graph at a `/scan` call a human triggers instead of a scheduler, drop the auto-execution step, and it's a research-and-alert tool — the same pre-trade research and risk workflow a desk already runs, alongside its terminal.

The autonomous loop is **one deployment mode** of the underlying system — not the only one.

VOLMIND

Thank you — questions welcome.

Full write-up, live demo recording, and test suite included with this submission.

TECHNICAL BRIEF	DEMO RECORDING	TESTS
`docs/TECHNICAL_BRIEF.md`	`docs/demo.mp4`	`pytest backend/tests — 33 passing`